Arquitectura técnica

Portal de Reclamaciones | Versión de entrega | 27 de octubre de 2026

1. Objetivo y vista lógica

Backend REST para consulta y evaluación de casos de reclamación, revisión humana, simulación de lotes de pago, auditoría, indicadores y exportación CSV. La descripción corresponde al entorno local verificado; no es una certificación de producción.

```
Cliente / Swagger UI
        |
Spring Security WebFlux - JWT Bearer / autoridades
        |
Controladores REST: Auth, Carga Excel, Evaluación, Parámetros,
Revisión, Consulta, Lotes, Reportes
        |
Servicios de negocio: reglas, decisiones, ANS, lotes, reportes, auditoría
        |
Spring Data JPA / Hibernate - Repositorios
        |
PostgreSQL: casos, catálogos, reglas, parámetros, seguimiento,
auditoría, caché de festivos y lotes

FestivosColombiaService <----> API externa Nager.Date
          |
          +---- caché local en PostgreSQL
```

Tecnología	Responsabilidad
Java 17 / Spring Boot 4.1.2	Runtime y configuración.
WebFlux / Netty	Capa HTTP de la API.
Spring Security + JWT	Autenticación Bearer sin sesión y autorización.
Spring Data JPA / Hibernate	Persistencia mediante repositorios.
PostgreSQL + Flyway	Datos y migraciones; V1 aplicada.
Springdoc OpenAPI 3.2.0	Swagger UI y especificación OpenAPI.
Nager.Date + caché	Festivos; caché local y error explícito ante fallo.

Componente	Responsabilidad
AuthController / SecurityConfig	Login, validación JWT y permisos de ruta.
ConsultaCasosController / RevisionCasosController	Consulta, detalle, toma, decisión y tracking.
EvaluacionController / ParametroController	Evaluación de reglas y parámetros.
LotesController	Lotes y confirmación de pago simulado.
ReportesController / ReportesService	Resumen y exportación CSV.
FestivosColombiaService	Festivos y auditoría de incidentes de API.

2. Flujos y API

Autenticación y consulta

El cliente envía credenciales a POST /api/auth/login. Un usuario válido recibe JWT y lo envía como Authorization: Bearer <token>. SecurityConfig valida el token y el claim scope habilita autoridades como SCOPE_ADMIN y SCOPE_ANALISTA.

Evaluación y revisión humana

El motor aplica 17 reglas por caso y persiste resultados para explicar la decisión y las reglas incumplidas. Los parámetros permiten ajustar umbrales. El analista toma un caso y puede emitir APROBAR u OBJETAR sujeto al estado, asignación y observación mínima configurada OBSERVACION_MIN_CARACTERES (20 caracteres en las pruebas).

Lotes y pago simulado

ADMIN o ANALISTA puede crear un lote; solo ADMIN puede confirmar el pago simulado. Estados y eventos de auditoría se registran. No hay transferencia bancaria real.

ANS y festivos

FestivosColombiaService usa la caché de festivos colombianos y consulta Nager.Date cuando es necesario. Si la API falla y no hay caché válida, devuelve SIN_FESTIVOS_VALIDOS con alerta y registra FALLO_API_FESTIVOS; no inventa fechas.

Área	Ruta	Acceso conocido
Login	POST /api/auth/login	Público
Consulta	GET /api/casos; GET /api/casos/{id}	JWT
Seguimiento	GET /api/casos/{id}/tracking	JWT
Revisión	POST /api/casos/{codigo}/tomar	ANALISTA
Decisión	PATCH /api/casos/{codigo}/estado	JWT + validaciones
Parámetros	PUT /api/parametros/**	ADMIN
Lotes	POST /api/lotes	ADMIN o ANALISTA
Pago simulado	PATCH /api/lotes/{id}/pagar	ADMIN
Reportes	GET /api/reportes/resumen; /api/reportes/exportar	JWT

NOTA Swagger UI está en /swagger-ui.html y OpenAPI JSON en /v3/api-docs. Actualmente la documentación es pública; los recursos de negocio siguen protegidos. Pruebas: OpenAPI 200 sin token y /api/casos 401 sin token.

3. Persistencia, verificación y operación

- Base de datos local observada: sura_reclamaciones. Flyway confirmó V1 aplicada y esquema al día.
- El insumo Excel se cargó una sola vez: 60 casos y catálogos. El motor registró 1.020 evaluaciones individuales (17 por cada caso).
- Tras ajustar MONTO_AUTO_BASICO, el resultado observado fue 30 AUTOMATICO y 30 MANUAL.
- FECHA_REFERENCIA se mantiene en 2026-10-14 según el escenario de prueba.
- Se confirmó un lote simulado de 31 casos; estado observado: 31 cerrados y 29 en bandeja.
- CSV validado: 60 filas con ID_Caso, Decision, Reglas_Incumplidas, Motivo, Prioridad, Estado_Actual, Fecha_Limite_ANS y Semaforo_ANS, en ese orden.

Compilar y ejecutar en Windows

```
.\mvnw.cmd clean verify
.\mvnw.cmd spring-boot:run
```

Requisitos observados: JDK 17, Maven Wrapper y PostgreSQL configurado en src/main/resources/application.properties. La última ejecución clean verify terminó en BUILD SUCCESS, con 2 pruebas y 0 fallos. Una prueba inicia el contexto; la otra simula fallo de la API de festivos con caché vacía y verifica el resultado y la auditoría.

Decisiones y límites

- No modificar Flyway V1 ni reimportar Excel para probar endpoints; los estados e historiales son parte de la demostración.
- Copiar resultados_reclamaciones_prueba.csv desde Descargas a la carpeta final de entrega.
- Antes de producción revisar perfil de Swagger, HTTPS, CORS, respaldos, monitoreo, rotación de secretos y ciclo de vida JWT; no están demostrados por estas pruebas.